



Getamazednow AI

A Getamazednow Company

MOCK / PORTFOLIO REFERENCE — TECHNICAL TRACK

Building the Real Thing

Build requirements, the Datadog implementation reference, and a phased, gated plan for an agent (or engineering team) to build the production infrastructure this demo mocks.

MOCK / SYNTHETIC DATA — NOT CONNECTED TO ANY LIVE SYSTEM

What we'll cover

05

Build requirements & dependencies

What has to be true before a real pilot workflow is instrumented

06

Datadog implementation reference

APIs, SDKs, metric namespace, dashboards, monitors, SLOs

07

If Claude Code were to build this

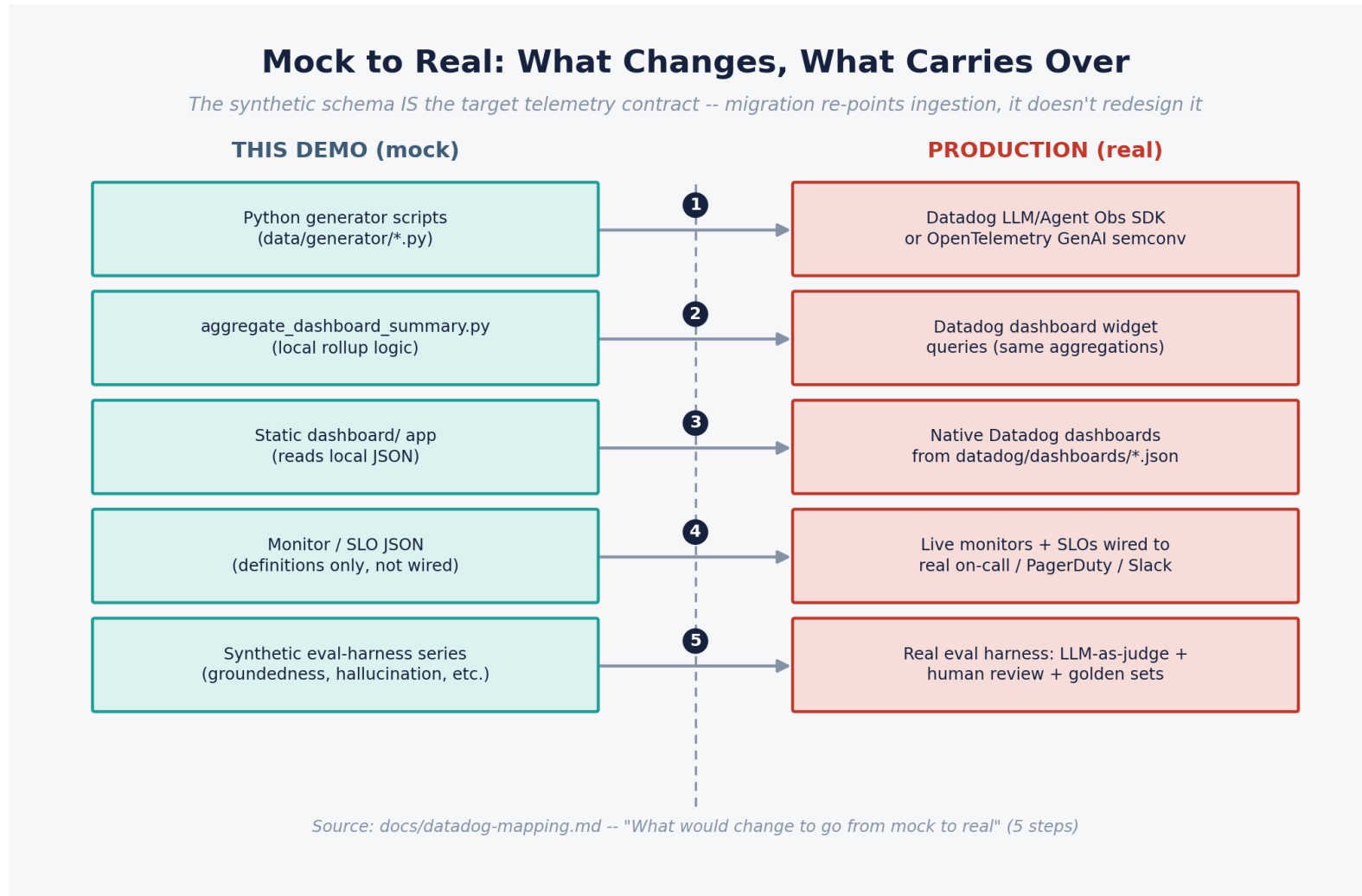
A phased, gated plan — with explicit human checkpoints

05

Build requirements & dependencies

What has to be true before this mock becomes a live, governed pilot

The synthetic schema is the target contract



Organisational & technical prerequisites

ORGANISATIONAL

- Named owner per accountability layer, agreed before instrumenting live traffic
- Risk tier definitions for the pilot workflow (low/medium/high — drives SLO targets and approvals)
- Redaction/classification/retention policy signed off by data governance and legal
- Executive sponsor for the cost-avoidance and ROI narrative

TECHNICAL

- Datadog org with LLM Observability + APM entitlements, admin access for dashboards-as-code import
- Instrumentation choice: Datadog LLM/Agent Obs SDK vs. OpenTelemetry GenAI semantic conventions
- Approved model/tool registry — sanctioned providers, models, tools and their risk classifications
- Tag taxonomy agreed before scaling past the pilot (high-cardinality raw IDs get expensive fast)
- On-call/alert-routing target to wire datadog/monitors and datadog/slos to

The one capability gap that matters most

A real evaluation harness

An LLM-as-judge pipeline, golden test sets, and a human-review sampling process. This is the single biggest capability gap between this demo and a defensible production system — groundedness, citation-accuracy, hallucination and regression scores are currently synthetic. Sensitive Data Scanner/DLP integration and a human-override concept in the product surface are the other two gaps in this phase.

This is a governance decision, not just an engineering task: Responsible AI/Risk must sign off on the evaluation methodology before its output gates any release. An agent can scaffold the plumbing to call an evaluation service — it should not select the methodology or curate golden sets.

06

Datadog implementation reference

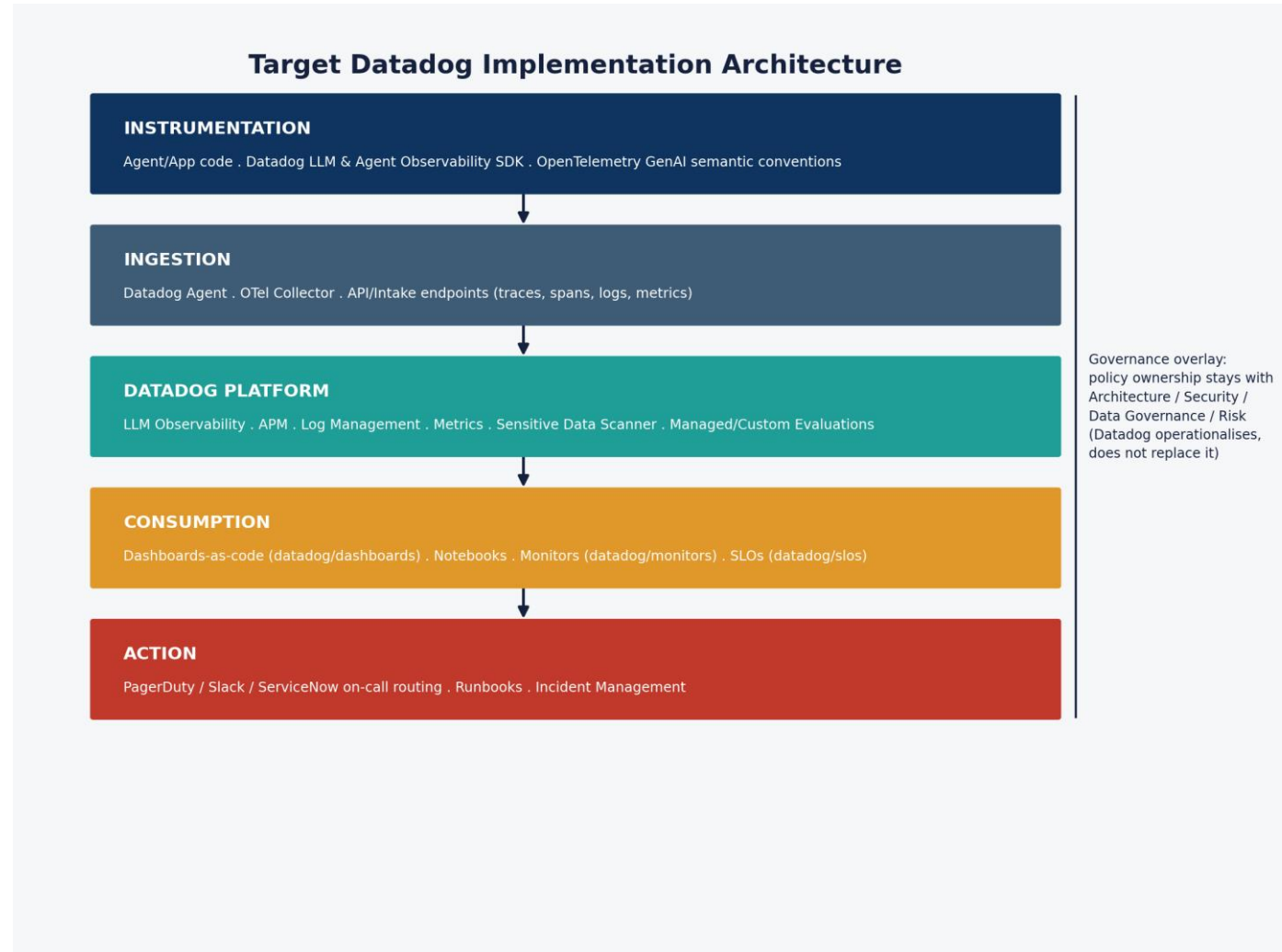
APIs, SDKs, the metric namespace, and the dashboards/monitors/SLOs already drafted in this repo

Which SDK / API to use

Approach	When to use it
Datadog LLM Observability SDK	Supported frameworks/providers (OpenAI, Anthropic, Bedrock, Azure OpenAI, LangChain, LangGraph)
Datadog Agent Observability SDK	Custom agent orchestration — planning, tool calls, loop spans
OpenTelemetry GenAI semantic conventions	Cross-vendor portability preferred, or unsupported frameworks
Manual spans / custom instrumentation	Anything auto-instrumentation doesn't cover (proprietary orchestration)

The synthetic span taxonomy in this repo (workflow / llm / retrieval / tool / guardrail) mirrors the addendum's recommended taxonomy — migrating the generator's field names into either SDK or OTel is a direct mapping exercise, not a redesign.

Target Datadog implementation architecture



genai.* — a starting convention

Metric	Type	Key tags
genai.workflow.count	count	outcome, use_case, tenant, channel, risk_tier
genai.workflow.cost_usd	distribution	use_case, tenant, cost_category
genai.llm.call.count	count	provider, model, status, finish_reason
genai.llm.tokens	distribution	provider, model, token_type
genai.tool.call.count	count	tool_name, risk_class, status
genai.guardrail.decision.count	count	policy_name, allow_block_escalate, reason_code
genai.eval.groundedness_score	gauge	use_case — from evaluation harness, not raw traces
genai.release.event	count	event_type, artefact, from_version, to_version

Full 20-metric namespace in docs/datadog-mapping.md and training/reading/05-datadog-implementation-reference.md.

Import path for dashboards, monitors & SLOs

Artefact	Contents	Import path
datadog/dashboards/*.json	7 templates, one per dashboard, status: implemented_in_demo	Dashboards API or datadog_dashboard Terraform resource
datadog/monitors/*.json	5 illustrative alerts (success rate, rate-limit, agent loop, high-risk-no-approval, data egress)	Monitors API or datadog_monitor Terraform resource
datadog/slos/*.json	SLO objects referencing the monitors as time-slices	SLO API, referencing monitor IDs above

Every monitor's notify target is a placeholder (@pagerduty-genai-oncall, @security-oncall). Re-pointing these at a real on-call rotation should be its own reviewed change — never bundled into the initial dashboards-as-code import.

07

If Claude Code were to build this

A phased, gated plan — scoped so an agent never holds live credentials or governance decisions alone

Ground rules before any agent touches this build

- **The synthetic schema is the target contract.** Don't redesign the telemetry schema; re-point it.
- **No live credentials in the agent's default context.** API keys and org access supplied per-phase, never committed.
- **Every phase ends with a human review gate,** not a merge-and-move-on — observability tooling operationalises controls, it doesn't replace ownership.
- **Additive, reversible changes.** Each phase is a mergeable, revertable unit of work.

Phase 0 — Preflight (human-only): named ownership, signed-off risk tier and redaction policy, staging Datadog org with a least-privilege service account. Not delegable to an agent.

Five phases, each with a named human gate

Phase	Agent-executable?	Roadmap horizon
0 — Preflight	No (human-only)	Weeks 0–2 prerequisite
1 — Instrumentation scaffold	Yes, sandboxed	Weeks 0–2
2 — Ingestion wiring	Yes, scoped credentials	Weeks 0–2
3 — Dashboards / monitors / SLOs	Yes, staging-only until gated	Weeks 3–6
4 — Governance / eval harness	Partially (plumbing only)	Weeks 7–10
5 — Scale / reusability	Yes, with acceptance test	Weeks 11–13

Each phase's prompt-ready task and human gate are written out in full in training/reading/06-claude-code-production-build-plan.md.

What an agent should never be handed unsupervised

Direct write access to production — Datadog org, on-call rotation, or alert-routing config; always a reviewed PR, never a direct autonomous API call.

Money-movement tooling credentials — beyond the sandboxed/staging environment used for testing.

The evaluation methodology decision — what counts as an acceptable eval or golden-set composition is a Responsible AI/Risk governance call, not an implementation detail.

Redaction/retention policy definition — the agent implements a policy humans already decided; it should not decide what's sensitive.

Sequencing summary & sources

A phased build converts this mock into a governed, production-grade observability platform — without ever asking an agent to make a policy decision on its own.

- Phases 1–3 (instrumentation, ingestion, dashboards) are the highest-leverage, lowest-risk agent work
- Phase 4 (governance/eval harness) is where human sign-off is non-negotiable
- Phase 5 (scale) only starts once a second use case can genuinely self-serve from the checklist

Sources: AI Observability Metrics Catalogue v1.0 • AI Observability — Datadog Implementation Addendum • docs/datadog-mapping.md • docs/roadmap.md • training/reading/06-claude-code-production-build-plan.md